

Consultas a varias tablas relacionadas en SQL con PostgreSQL

Esta lectura corresponde al Aprendizaje Esperado 5 del Módulo 6 del plan formativo de Análisis de Datos: Construir consultas a varias tablas relacionadas utilizando el modelo de datos para resolver una necesidad de información acorde al lenguaje SQL. El objetivo es entregar una base conceptual y práctica sólida que permita utilizar correctamente las consultas en SQL con especial foco en PostgreSQL.

Modelo entidad-relación y relaciones entre tablas

El modelo entidad-relación (ER) es una representación abstracta de los datos y de las relaciones entre ellos. Cada entidad se corresponde con una tabla en la base de datos. Las relaciones se representan mediante claves primarias y claves foráneas, que permiten establecer conexiones lógicas entre diferentes tablas. En PostgreSQL, las claves foráneas aseguran la integridad referencial entre tablas.

Consultas a tablas relacionadas

Las consultas a varias tablas son fundamentales para obtener información combinada. El uso de joins permite vincular registros en función de condiciones de igualdad o de relaciones definidas en el modelo de datos.

Vinculación de datos

Los **joins** permiten combinar información de múltiples tablas

Condiciones de igualdad

Se establecen relaciones basadas en **claves primarias y foráneas**

Modelo de datos

Las consultas respetan la **estructura relacional** definida

Tipos de JOIN en SQL y PostgreSQL

01

INNER JOIN

Devuelve solo las filas con coincidencias en ambas tablas.

02

LEFT JOIN

Devuelve todas las filas de la tabla izquierda y las coincidentes de la tabla derecha.

03

RIGHT JOIN

Devuelve todas las filas de la tabla derecha y las coincidentes de la tabla izquierda.

04

FULL OUTER JOIN

Devuelve todas las filas cuando hay coincidencias en una u otra tabla.

Operaciones de conjunto en SQL

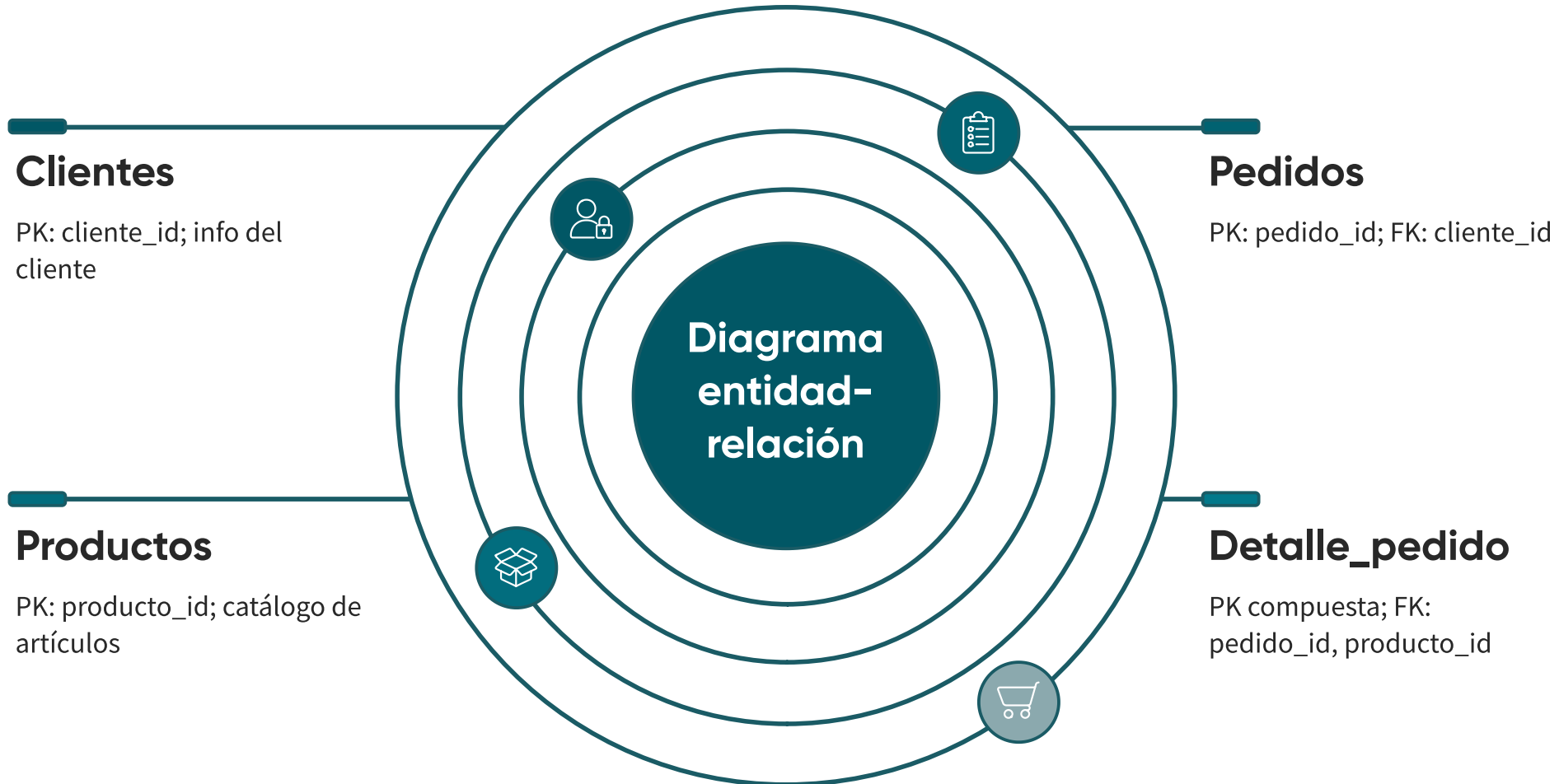
PostgreSQL soporta operaciones de conjunto que permiten combinar resultados de múltiples consultas:

- **UNION:** Combina resultados eliminando duplicados (o UNION ALL para mantenerlos).
- **INTERSECT:** Devuelve los registros que existen en ambos conjuntos.
- **EXCEPT:** Devuelve los registros que están en el primer conjunto pero no en el segundo.



Representación de las relaciones entre tablas

Las relaciones entre tablas se representan gráficamente en diagramas entidad-relación (ERD). Estos diagramas permiten visualizar cómo se conectan las tablas mediante claves primarias y foráneas.



Parte práctica: ejemplos en PostgreSQL

Consideremos las siguientes tablas de ejemplo:

- `clientes(id_cliente, nombre, ciudad)`
- `pedidos(id_pedido, id_cliente, fecha)`
- `productos(id_producto, nombre, precio)`
- `detalle_pedido(id_pedido, id_producto, cantidad)`

Ejemplos prácticos de consultas

Ejemplo 1: Obtener todos los pedidos con datos de cliente

```
SELECT p.id_pedido, c.nombre, p.fecha  
FROM pedidos p  
INNER JOIN clientes c ON p.id_cliente = c.id_cliente;
```

Ejemplo 2: Identificar clientes sin pedidos

```
SELECT c.id_cliente, c.nombre  
FROM clientes c  
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente  
WHERE p.id_pedido IS NULL;
```

Ejemplo 3: Productos no vendidos

```
SELECT pr.id_producto, pr.nombre  
FROM productos pr  
LEFT JOIN detalle_pedido dp ON pr.id_producto = dp.id_producto  
WHERE dp.id_pedido IS NULL;
```


Ejemplo 4: Operaciones de conjunto

```
-- Clientes que hicieron pedidos en enero
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31'

-- Clientes que hicieron pedidos en febrero
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28'

-- Clientes que compraron en ambos meses
INTERSECT

-- Clientes que compraron en enero pero no en febrero
EXCEPT;
```

Ejercicios propuestos

1. Crear una consulta que muestre el total de productos comprados por cada cliente.
2. Obtener una lista de clientes y la cantidad de pedidos realizados, incluyendo aquellos que no tienen pedidos.
3. Usar UNION para combinar en una sola lista los clientes de dos ciudades distintas.
4. Identificar los productos vendidos en todos los pedidos (usando INTERSECT).
5. Representar el modelo de datos con un diagrama ER que incluya clientes, pedidos, productos y detalle_pedido.

Tipos de JOIN en SQL y PostgreSQL con ejemplos

INNER JOIN

El INNER JOIN devuelve únicamente las filas que cumplen la condición de coincidencia entre las tablas. Es el más utilizado cuando queremos obtener información que está relacionada en ambas tablas.

Ejemplo: obtener todos los pedidos con el nombre del cliente:

```
SELECT p.id_pedido, c.nombre, p.fecha  
FROM pedidos p  
INNER JOIN clientes c ON p.id_cliente = c.id_cliente;
```

LEFT JOIN

El LEFT JOIN devuelve todas las filas de la tabla izquierda aunque no existan coincidencias en la tabla derecha. Si no hay coincidencia, los valores de la tabla derecha se rellenan con NULL.

Ejemplo: listar todos los clientes y sus pedidos, incluso si no tienen pedidos:

```
SELECT c.id_cliente, c.nombre, p.id_pedido, p.fecha  
FROM clientes c  
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

RIGHT JOIN y FULL OUTER JOIN

RIGHT JOIN

El RIGHT JOIN es el inverso del LEFT JOIN. Devuelve todas las filas de la tabla derecha y las coincidencias de la tabla izquierda. Los valores de la tabla izquierda que no coinciden se muestran como NULL.

Ejemplo: mostrar todos los pedidos y, en caso de que no tengan cliente asociado (inconsistencia en datos), mostrar NULL:

```
SELECT p.id_pedido, c.nombre, p.fecha  
FROM pedidos p  
RIGHT JOIN clientes c ON p.id_cliente = c.id_cliente;
```

FULL OUTER JOIN

El FULL OUTER JOIN devuelve todas las filas de ambas tablas, tanto las coincidentes como las no coincidentes. Donde no hay coincidencia, los valores se completan con NULL.

Ejemplo: combinar clientes y pedidos, incluyendo aquellos sin coincidencia en ambos lados:

```
SELECT c.id_cliente, c.nombre, p.id_pedido, p.fecha  
FROM clientes c  
FULL OUTER JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

Operaciones de conjunto en SQL con ejemplos

UNION y UNION ALL

UNION permite combinar resultados de dos consultas eliminando duplicados, mientras que UNION ALL mantiene los duplicados.

Ejemplo: listar todos los clientes de las ciudades 'Santiago' y 'Valparaíso':

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Santiago'  
UNION  
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Valparaíso';  
  
-- Con UNION ALL se mantienen duplicados
```

INTERSECT

INTERSECT devuelve únicamente las filas que aparecen en ambas consultas.

Ejemplo: clientes que han comprado tanto en enero como en febrero:

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31'  
INTERSECT  
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28';
```

EXCEPT y EXCEPT ALL

EXCEPT devuelve las filas de la primera consulta que no aparecen en la segunda. EXCEPT ALL permite conservar duplicados.

Ejemplo: clientes que compraron en enero pero no en febrero:

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31'  
EXCEPT  
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28';
```

Casos de negocio prácticos

A continuación se presentan casos de negocio contextualizados en distintas industrias para aplicar el uso de consultas a varias tablas relacionadas en PostgreSQL. Estos ejemplos permiten entender cómo se utilizan los JOINS y las operaciones de conjunto en escenarios reales.



Caso 1: Retail – Ventas en tienda

En una empresa de retail existen las siguientes tablas: clientes(id_cliente, nombre, ciudad), pedidos(id_pedido, id_cliente, fecha), productos(id_producto, nombre, precio), detalle_pedido(id_pedido, id_producto, cantidad). Se desea obtener un reporte de ventas por cliente, mostrando el total gastado en productos.

```
SELECT c.nombre,  
SUM(dp.cantidad * pr.precio)  
AS total_gastado  
FROM clientes c  
INNER JOIN pedidos p ON  
c.id_cliente = p.id_cliente  
INNER JOIN detalle_pedido dp  
ON p.id_pedido = dp.id_pedido  
INNER JOIN productos pr ON  
dp.id_producto =  
pr.id_producto  
GROUP BY c.nombre;
```



Caso 2: Banca – Clientes y transacciones

En un banco existen las tablas: clientes(id_cliente, nombre, ciudad), cuentas(id_cuenta, id_cliente, tipo_cuenta), transacciones(id_transaccion, id_cuenta, fecha, monto). Se desea identificar los clientes que han realizado depósitos en enero pero no en febrero.

```
SELECT c.id_cliente, c.nombre  
FROM clientes c  
INNER JOIN cuentas cu ON  
c.id_cliente = cu.id_cliente  
INNER JOIN transacciones t ON  
cu.id_cuenta = t.id_cuenta  
WHERE t.fecha BETWEEN '2023-  
01-01' AND '2023-01-31'  
EXCEPT  
SELECT c.id_cliente, c.nombre  
FROM clientes c  
INNER JOIN cuentas cu ON  
c.id_cliente = cu.id_cliente  
INNER JOIN transacciones t ON  
cu.id_cuenta = t.id_cuenta  
WHERE t.fecha BETWEEN '2023-  
02-01' AND '2023-02-28';
```



Caso 3: Salud – Pacientes y consultas médicas

En una clínica se tienen las tablas: pacientes(id_paciente, nombre, edad), medicos(id_medico, nombre, especialidad), consultas(id_consulta, id_paciente, id_medico, fecha, diagnostico). Se requiere obtener una lista de pacientes junto con la cantidad de consultas realizadas con médicos de la especialidad 'Cardiología'.

```
SELECT p.nombre AS paciente,  
COUNT(c.id_consulta) AS  
total_consultas  
FROM pacientes p  
INNER JOIN consultas c ON  
p.id_paciente = c.id_paciente  
INNER JOIN medicos m ON  
c.id_medico = m.id_medico  
WHERE m.especialidad =  
'Cardiología'  
GROUP BY p.nombre;
```

Ejercicios guiados paso a paso

En esta sección se presentan ejercicios prácticos que los estudiantes pueden seguir paso a paso. Cada ejercicio incluye un enunciado, la construcción guiada de la consulta SQL y el resultado esperado.

Ejercicio 1: Obtener clientes y su número de pedidos

Objetivo: Listar todos los clientes junto con la cantidad de pedidos que han realizado.

Paso 1: Seleccionar la tabla de clientes.

```
SELECT id_cliente, nombre FROM clientes;
```

Paso 2: Hacer un JOIN con la tabla de pedidos.

```
SELECT c.id_cliente, c.nombre, p.id_pedido
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

Paso 3: Usar COUNT() y GROUP BY para contar pedidos por cliente.

```
SELECT c.id_cliente, c.nombre, COUNT(p.id_pedido) AS total_pedidos
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nombre;
```

Ejercicio 2: Identificar productos más vendidos

Objetivo: Mostrar los productos con la cantidad total vendida.

Paso 1: Seleccionar los productos.

```
SELECT id_producto, nombre FROM productos;
```

Paso 2: Relacionar con detalle_pedido.

```
SELECT pr.id_producto, pr.nombre, dp.cantidad
FROM productos pr
INNER JOIN detalle_pedido dp ON pr.id_producto = dp.id_producto;
```

Paso 3: Sumar cantidades y agrupar por producto.

```
SELECT pr.id_producto, pr.nombre, SUM(dp.cantidad) AS total_vendido
FROM productos pr
INNER JOIN detalle_pedido dp ON pr.id_producto = dp.id_producto
GROUP BY pr.id_producto, pr.nombre
ORDER BY total_vendido DESC;
```

Ejercicio 3: Clientes que compraron en dos periodos distintos

Objetivo: Obtener clientes que compraron tanto en enero como en febrero.

Paso 1: Listar clientes que compraron en enero.

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31';
```

Paso 2: Listar clientes que compraron en febrero.

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28';
```

Paso 3: Usar INTERSECT para obtener clientes comunes.

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31'
INTERSECT
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28';
```

Ejercicio 4: Clientes sin pedidos (LEFT JOIN + NULL)

Objetivo: Identificar clientes que aún no han realizado ningún pedido.

Paso 1: Hacer LEFT JOIN entre clientes y pedidos.

```
SELECT c.id_cliente, c.nombre, p.id_pedido
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

Paso 2: Filtrar los que no tienen pedidos asociados.

```
SELECT c.id_cliente, c.nombre
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
WHERE p.id_pedido IS NULL;
```

Ejercicio 5: Comparar clientes de dos ciudades (UNION)

Objetivo: Generar una lista única de clientes de Santiago y Valparaíso.

Paso 1: Seleccionar clientes de Santiago.

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Santiago';
```

Paso 2: Seleccionar clientes de Valparaíso.

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Valparaíso';
```

Paso 3: Combinar con UNION para evitar duplicados.

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Santiago'
UNION
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Valparaíso';
```